Compilation Project

Gwenaël Delaval, Guillaume Iooss

gwenael.delaval@univ-grenoble-alpes.fr
guillaume.iooss@inria.fr

Université Grenoble Alpes

December 15th, 2025

Contents

1 Organization

2 Compiler Project

Contents

1 Organization

2 Compiler Project

Practical Organization

- Mon. 15/12 — 9h30-10h30 Presentation of the project
- Fri. 19/12, 10/01, 17/01 — 18h weekly report (mid-project submission on 13 and clone of the git repository)
- Thu. 22/01 — 12h Final project submission and clone of the git repository.
- Fri. 23/01 — 09h-17h Project defense

Main informations on the Moodle page:
http://im2ag-moodle.univ-grenoble-alpes.fr/course/view.php?id=234

To join us:

- by e-mail (add the two email addresses and the string "[M1-Compil]" in the mail subject to maximize our reactivity!)
- by mattermost: (maybe not available this year?)

Weekly Organization

Team organization

Highly recommended: every day, ≈10 minutes (short) meetings at fixed hour.

For each task:

- - new task: description of task
  - existing task: status of task (work done, work remaining...)
- people assigned on task
- expected time of completion

Assign the "scribe" role: takes log of meeting (must not change during week).

Weekly Organization

Team organization

Highly recommended: every day, ≈10 minutes (short) meetings at fixed hour.

For each task:

- - new task: description of task
  - existing task: status of task (work done, work remaining...)
- people assigned on task
- expected time of completion

Assign the "scribe" role: takes log of meeting (must not change during week).

Weekly report is the list of all five logs of the week.

Group Organization

Subscribe on moodle ASAP, you can then choose a team. Then send us an email with:

- team members (4-6 people)
- team name :-)
- team "coordinator" (recommended)
- programming language (we recommend OCaml or Java)
- project name on the gricad gitlab (+ add us: delavalg & [TODO login GI], as members)

http://im2ag-moodle.univ-grenoble-alpes.fr/course/view.php?id=234
⇒ watch the forum for info (time slots, lectures...)

Grade

Grade will take into account:

- Respect of requirements
- Code:
  - features
  - writing quality (adhere to coding conventions, write well-documented and modular programs)
  - robustness
  - test suite
- Report:
  - "User documentation": what we need to know to use your compiler
  - "Developer documentation": what a new developer would need to know, including documentation of test suite
- Demo:
  - 20 min. presentation (technical & usage/features).
  - 10 min. questions
- Team management (weekly reports,. . .)

Cheating/plagiarism

- You must write all the code you hand in, except for code that was provided. You are not allowed to look at anyone else's solution, but you may discuss your assignments with other students.
- We use plagiarism detection tools.
- We will follow you during the project.

Project setup

- Download `archive.tar.gz` from moodle and untar it (`tar zxvf archive.tar.gz`).
- All the instructions and documentation for the project is accessible from `doc/index.html`.
- Depending on the language you will use for the project, you will start with the code given in OCaml or Java.
- See README for the organization of the archive.

Git setup

To set-up your git repository on the gricad-gitlab, see the details in `archive/doc/index.html` and `archive/doc/tutorial_git.html`.

- Register on https://gricad-gitlab.univ-grenoble-alpes.fr/
- Register a new project
- Add files to your own project
- Then everyone can get a copy

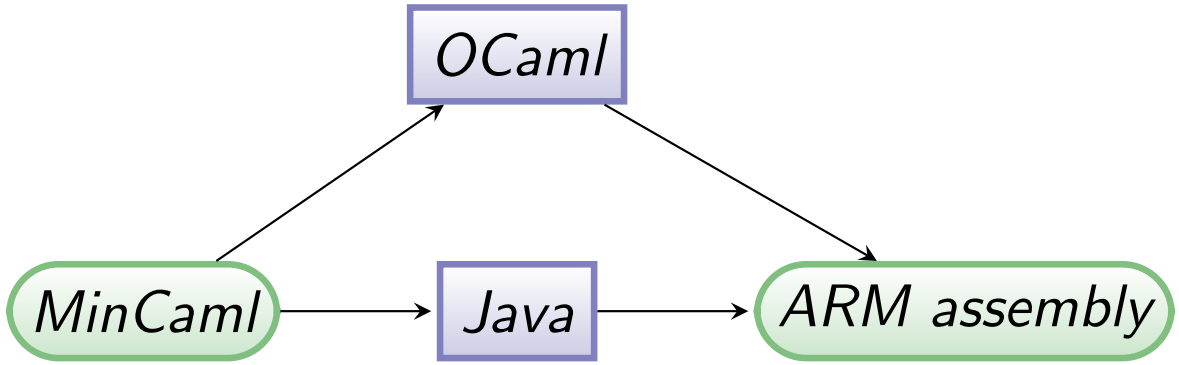
Contents

1 Organization

2 Compiler Project

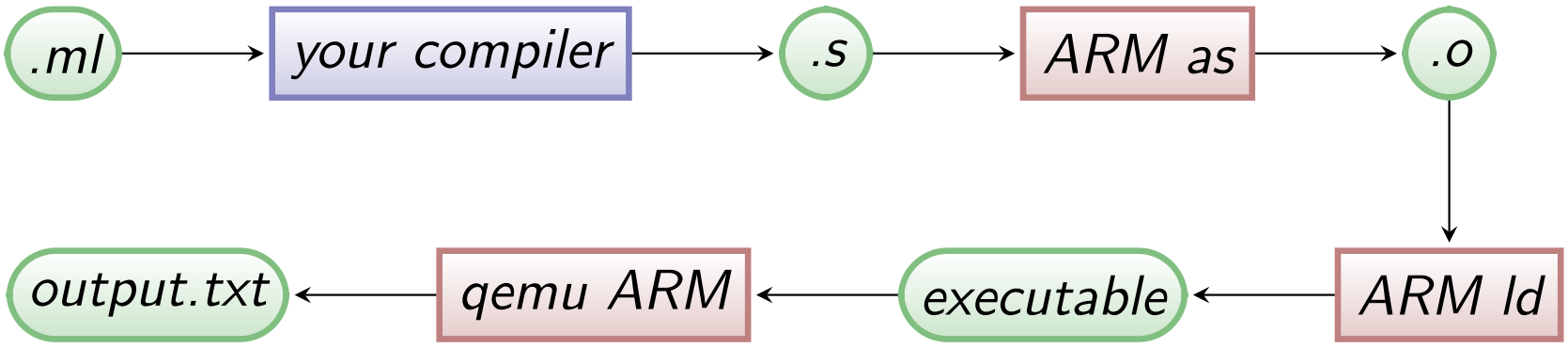
The Compiler

Skeletons given in OCaml or Java with parser and minimal tests.



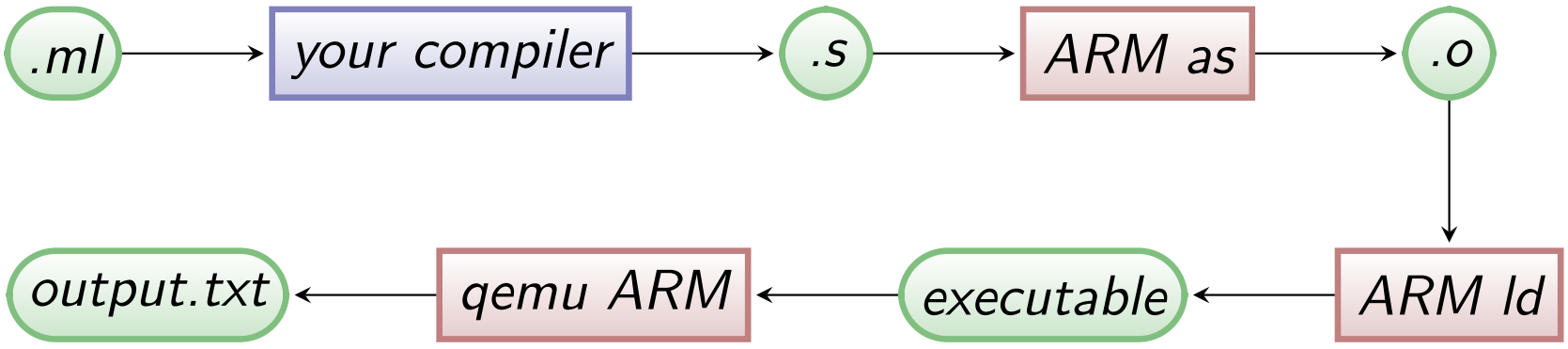
Toolchain

Compiler Toolchain:

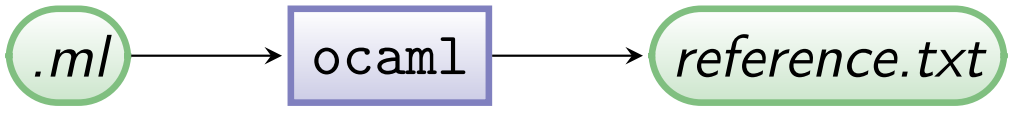


Toolchain

Compiler Toolchain:



Comparison:



Warning: `ocaml` interprets the code (no compilation).

Toolchain Example

```
$ ../min-caml gcd.ml -o gcd.s
$ arm-none-eabi-as -o gcd.o gcd.s libmincaml.S
$ arm-none-eabi-ld -o gcd gcd.o
$ qemu-arm ./gcd
```

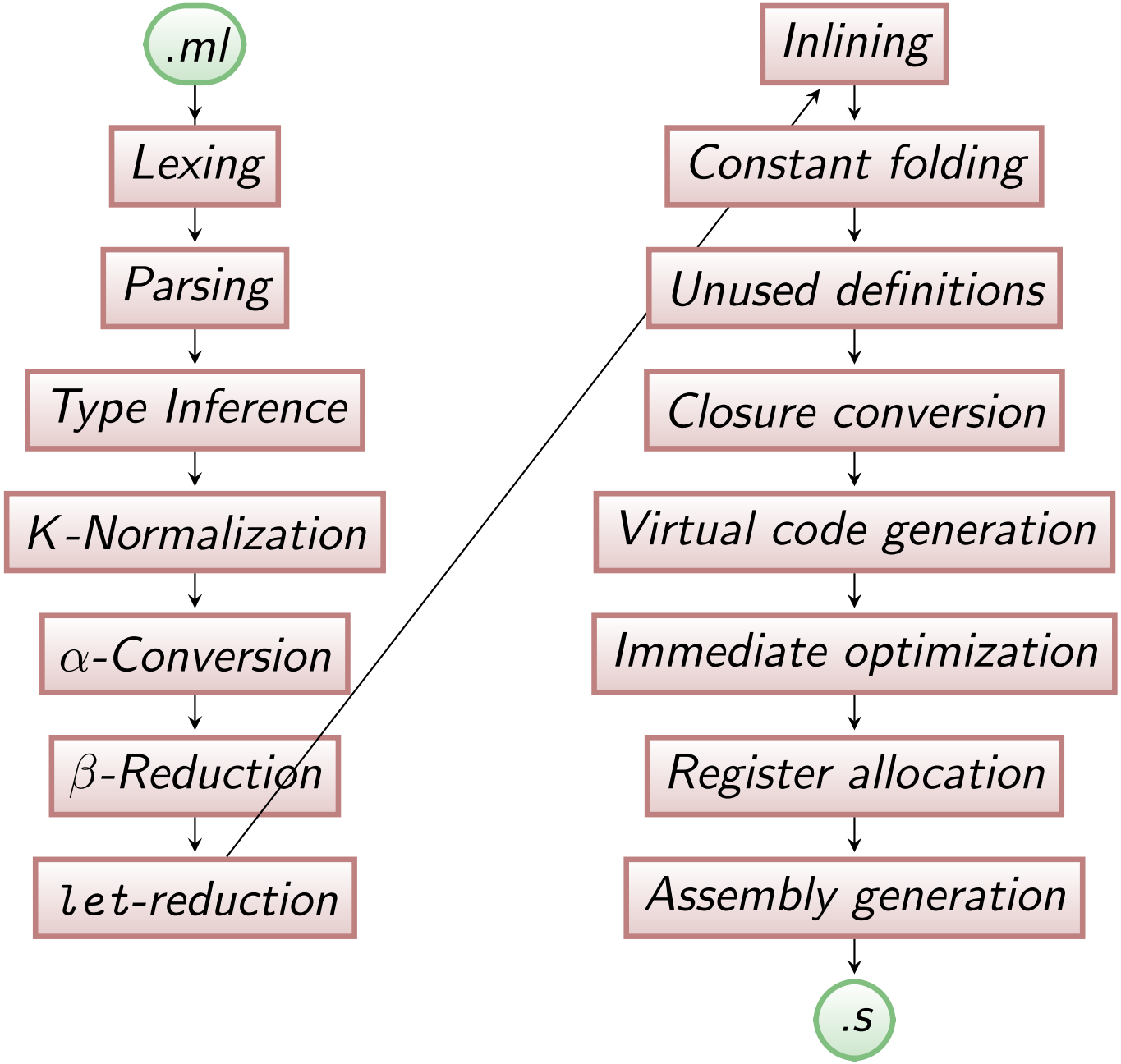
See in the archive for other examples.

GCD Example

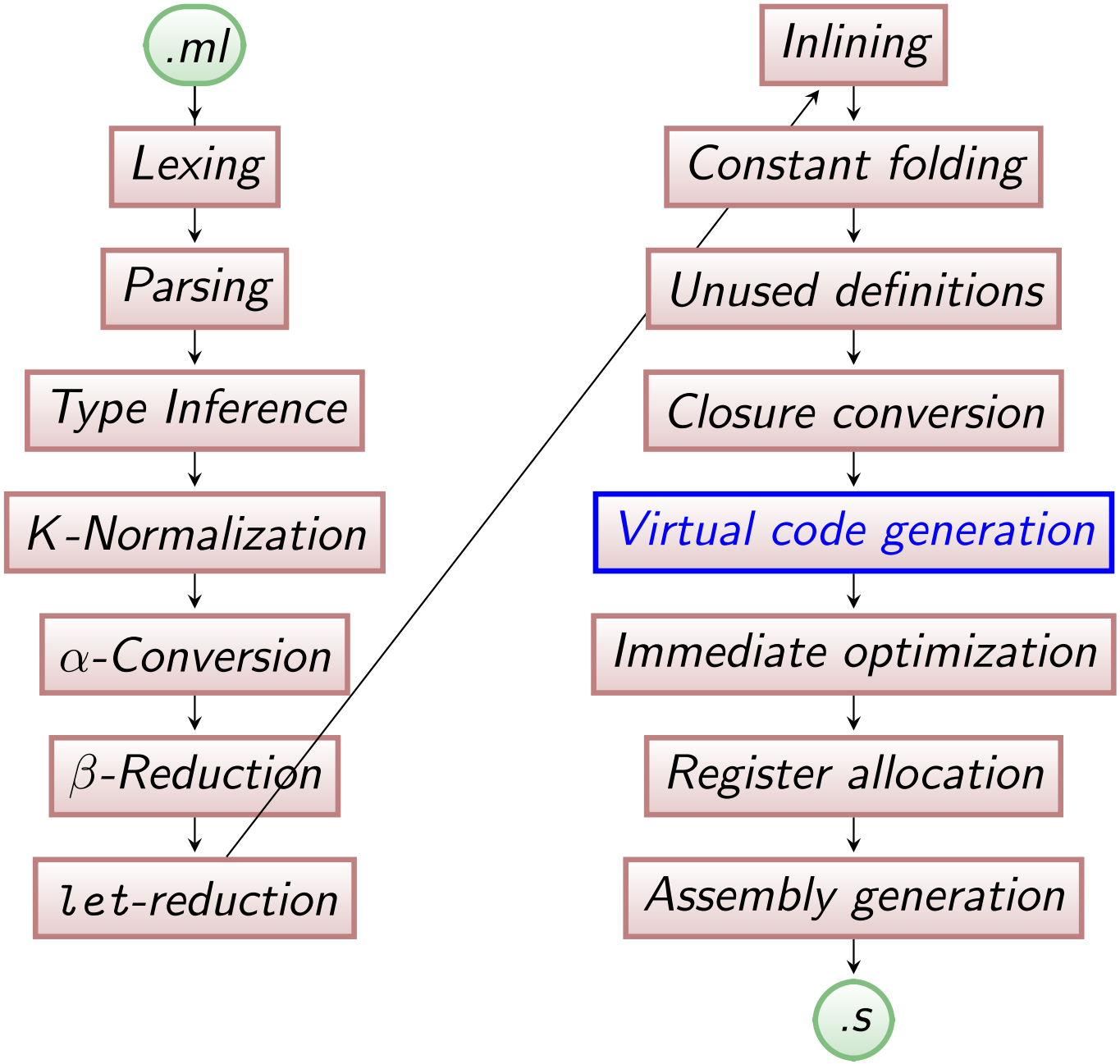
```
let rec gcd m n =
  if m = 0 then n else
  if m <= n then gcd m (n - m) else
  gcd n (m - n)
```

```
        .text
gcd.7:
        cmp   r0, #0
        bne   be_else.17
        mov   r0, r1
        bx    lr
be_else.17:
        cmp   r0, r1
        bgt   ble_else.18
        sub   r1, r1, r0
        b     gcd.7
        nop
ble_else.18:
        sub   r0, r0, r1
        mov   r14, r1
        mov   r1, r0
        mov   r0, r14
        b     gcd.7
        nop
```

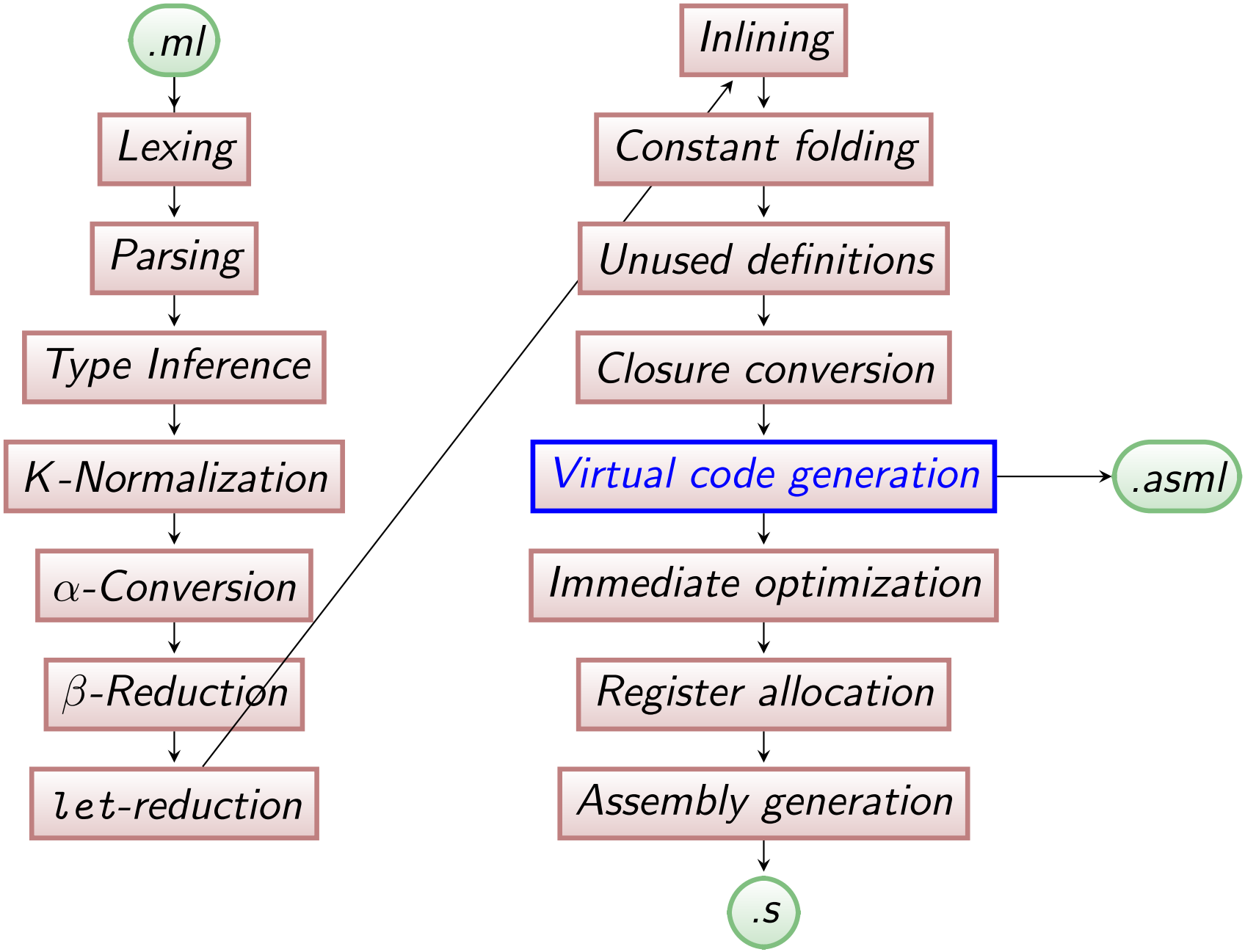
Compiler Passes



Compiler Passes



Compiler Passes



Tasks and Incremental Development

Decide what you want to do!

- Front-end (35%)
- Type inference and type checking (15%)
- Back-end (35%)
- Testing (15%)

Each task can be subdivided furthermore.

- Don't work one pass at a time, but one feature at a time
- Integrate ASAP

Incremental development

- simple arithmetic expression
- call to external functions
- if-then-else
- functions (let rec)
- tuples and arrays
- closures

The ASML intermediate representation

ASML is at the interface between front-end and back-end.
It is a *Virtual Machine Code*.

Features

- values stored in temporaries ("registers")
- functions represented by labels
- no nested definitions
- memory accesses are explicit
- if-then-else constructs
- operators for memory allocations

The ASML intermediate representation

ASML is at the interface between front-end and back-end.
It is a *Virtual Machine Code*.

ASML example

```
let _ =
  let size = 2 in
  let init = 0 in
  let arr = call _min_caml_create_array size init in
  let i0 = 0 in
  let v0 = 1 in
  let tu = mem(arr + i0) <- v0 in
  let i1 = 1 in
  let v1 = mem(arr + i0) in
  mem(arr + i1) <- v1
```

The ASML intermediate representation

- We provide an interpeter for ASML. You can test the front-end independently of the back-end!
  - Compare the behavior of the ASML program with the behavior of the source file (using ocaml)
  - But you need to output ASML (easy)
- you can test the back-end independently of the front-end too
  - But you need to be able to parse ASML (more difficult!). Maybe a simpler representation (JSON?)
- In any case, you will have to write a datatype that encode ASML programs.

The Command Line

We will use our test suite on your compilers.

Return value

- 0 on success
- 1 on error, plus error message on stderr

Required command-line options

-o <file> : output file

-h : display help

-v : display version

-t : only type checking

-asml : print ASML

The Command Line

We will use our test suite on your compilers.

Return value

- 0 on success
- 1 on error, plus error message on stderr

Required command-line options

-o <file>	:	output file
-h	:	display help
-v	:	display version
-t	:	only type checking
-asml	:	print ASML
-my-opt	:	you can add personal options (optimizations, etc.)

Git Advices

We will clone your git repositories on Friday 10th January at 6pm.

We will only consider the **main** branch!

`make` must compile your compiler

`make test` must launch your test suite

Git Advices

We will clone your git repositories on Friday 10th January at 6pm.

We will only consider the **main** branch!

`make` must compile your compiler

`make test` must launch your test suite

- we advise you to work on development branches

```
$ git checkout -b dev
$ git checkout -b <login>
```

- push on `dev` to share development with others

```
$ git checkout dev
$ git merge <login>
$ git push
$ git checkout <login>
```

- push on `main` only when it's safe

main must always be in a working state!

Compressed TODO List

- Register on moodle (COMPPROJ course)
- Register project for your team on `https://gricad-gitlab.univ-grenoble-alpes.fr/`
- Coordinator sends email (with everyone in CC) to `gwenael.delaval@univ-grenoble-alpes.fr` and `guillaume.iooss@inria.fr`
- Read the documentation, including the min-caml article
- Note the other available information moodle
- Watch for news on moodle forum (esp. time&place of meetings)
- Start organization! (stand-up meetings, etc.)